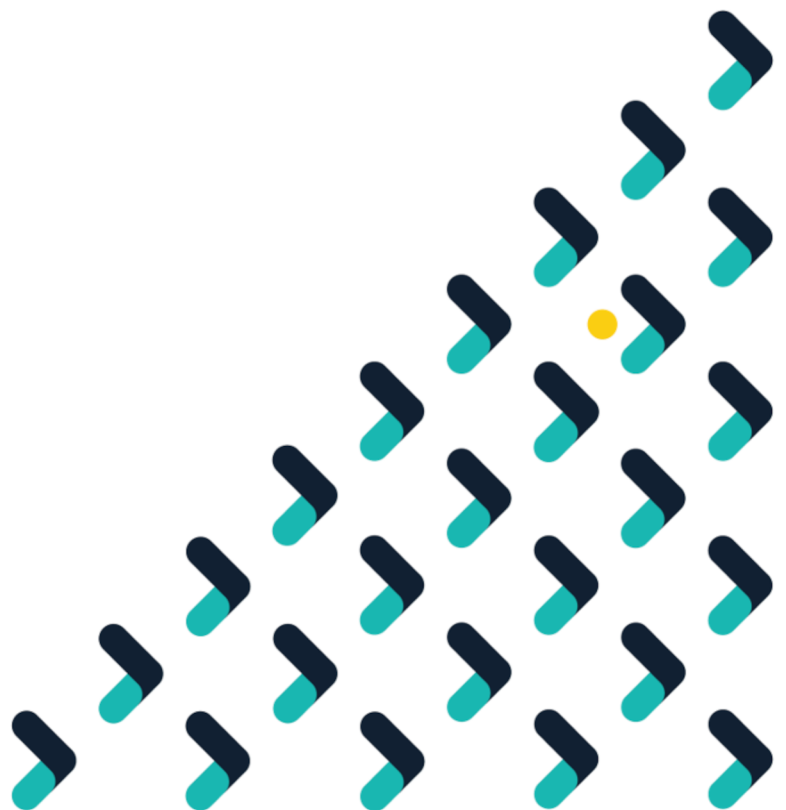




FO2-Programmierung

Forterro Deutschland Abas GmbH

2026-06-17



Inhaltsverzeichnis

FO2-Programmierung	1
Administrative Einstellungen	2
Installation	2
Konfiguration der Abas Tools für FO2	2
EDP-Verbindungsparameter/SSH-Login	2
EDP-Logging	3
Compilereinstellungen	3
Infrastruktur für FO2-Programmierung	3
FO2-Datei	5
FO2-Datei erzeugen	5
FO2-Datei erweitern	6
Geänderte FO-/FO2-Dateien zum Server hochladen	7
FO2-Datei (FO2-Programm) ausführen	7
FO2-Datei generieren	7
Editoreigenschaften	9
VAR-Tag	9
BLOCK-Tag	9
TODO	10
FIXME	10
META-Tag	11
FO2-Sprachkomponenten	12
Kommentare	12
Variablen	12
Prozeduren	12
if-Bedingungen	13
switch-case	14
while	15
do while	16
.break	17
.continue	17
.return	18
Templates	19
Datensätze selektieren	20
Kundendatensätze selektieren	20
Artikeldatensätze selektieren	20
Menü verwenden	22
Textbox	24
etf-Datei importieren	25
Diffs	26
Weitere Beispiele	27
Array in FO2 verwenden	27
Zugriff auf die Variable actionname (deutsch: kaktname)	27

FO2-Programmierung

FO2-Programme werden mit dem FO Developer entwickelt. Dieser ist Bestandteil der [Abas Tools](#). Die mit dem FO Developer erzeugten Programme haben die Endung *.F02. Das Programmieren wird durch Auswahllisten, automatische Textvervollständigung und Vorlagen erleichtert. Das FO2-Programm kann durch Verwendung von Prozeduren modular aufgebaut werden. Der FO Developer steuert den Programmablauf mit Schleifen und Bedingungen.

Administrative Einstellungen

Installation

Laden Sie die passende Abas Tools-Version von der Abas Homepage herunter. Entpacken Sie die ZIP-Datei in einen Ordner Ihrer Wahl. Abas Tools wird im Unterverzeichnis **abas-tools** entpackt. Dieses Unterverzeichnis enthält unter anderem die Datei **abasTools.exe**.

Start von Abas Tools:

1. Öffnen Sie das Verzeichnis **abas-tools**.
2. Führen Sie die Datei **abasTools.exe** aus.
3. Verwenden Sie einen definierten Workspace.



Der **Workspace** ist ein definiertes Verzeichnis auf Ihrem Rechner.

Konfiguration der Abas Tools für FO2

EDP-Verbindungsparameter/SSH-Login

Konfigurieren Sie EDP-Verbindungsparameter und das SSH-Login für den Up- und Download der FO-Files.

Führen Sie folgende Schritte aus:

1. Wählen Sie das Menü [Window] [Preferences] um das Fenster Preferences zu öffnen
2. Öffnen Sie die Struktur [Abas Tools Suite] [Connector JEDP] [Connection Configuration].
3. Klicken Sie den Button **Add** um das Fenster **Edit Connection** zu öffnen.
4. Tragen Sie im Register **Connection Properties** folgende Informationen ein:
 - a) Connection Name:* Verbindungsname, z.B. i7erp15@schulung
 - b) Host: Servername, z.B. schulung
 - c) Client: Mandantennamen, z.B. i7erp15
 - d) Port: EDP-Port, default ist 6550 eingetragen
 - e) Password: Das Benutzer-Passwort
5. Wechseln Sie in das Register **Client Options**.
6. Tragen Sie folgende SSH-Login-Informationen ein:
 - a) User Name: Linux-Benutzer, z.B. schulvajoPassword: Passwort des Linux-Benutzers
7. Klicken Sie auf **Ok**.

Die Einträge werden gespeichert. Im Fenster Preferences wird eine neue Zeile mit den EDP-Verbindungsparametern eingetragen. Markieren Sie die Zeile.
8. Klicken Sie auf **Test** um die EDP-Verbindung zum Mandanten zu testen.
9. Schließen Sie nach erfolgreichem Test das Fenster Preferences.

EDP-Logging

Das EDP-Logging ist standardmäßig deaktiviert. Sie können das EDP-Logging aktivieren.



Aktivieren Sie das EDP-Logging nur im Fehlerfall, da sonst die komplette EDP-Session protokolliert wird.

Führen Sie folgende Schritte aus:

1. Wählen Sie das Menü [Window] [Preferences] um das Fenster Preferences zu öffnen.
2. Öffnen Sie die Struktur [Abas Tools Suite] [Connector JEDP] [Connection Configuration].
3. Markieren Sie die Zeile mit den EDP-Verbindungsparametern, deren EDP-Logging Sie aktivieren möchten.
4. Klicken Sie auf **Edit** um das Fenster **Edit Connection** zu öffnen.
5. Markieren Sie das Feld Logging EDP communication und klicken Sie **Ok**.

Compilereinstellungen

Beim Speichern der FO2-Datei wird eine FO-Datei compiliert. Damit das Compilieren automatisch ausgeführt wird, müssen Sie den FOP-Generator wie folgt aktivieren:

1. Wählen Sie das Menü [Window] [Preferences], um das Fenster Preferences zu öffnen.
2. Öffnen Sie die Struktur [Fop2] [Generator], um das Fenster Generator zu öffnen.
3. Markieren Sie das Feld **Enable FOP Generator** und klicken Sie **Ok**.

Infrastruktur für FO2-Programmierung

Die Basis für die FO2-Programmierung bildet ein FO Project. Im FO Project können Sie über eine SSH-Verbindung auf die Arbeitsbereiche des angemeldeten Benutzers zugreifen. Sie können leere oder komplette Arbeitsbereiche herunterladen. Sie können aus einem Arbeitsbereich eine oder mehrere FO2-Dateien herunterladen. Sie können im FO Project heruntergeladene FO2-Dateien ändern oder neue FO2-Dateien erzeugen. Sie müssen die geänderten oder neu erzeugten FO2-Dateien in den Arbeitsbereich hochladen, damit sie ausgeführt werden können.

FO Project erzeugen und administrieren

Um ein FO Project zu erzeugen Führen Sie folgende Schritte aus:

1. Wechseln Sie in Abas Tools in das Register **PackageExplorer**.
2. Öffnen Sie das Kontextmenü und wählen Sie [New] [Other...] [FO Project].
3. Klicken Sie auf **Next**, um das Fenster **New FO Project** zu öffnen.
4. Tragen Sie in das Feld **Project Name** den Projektnamen ein, z.B. FirstFoProject und klicken Sie auf **Finish**.



Damit Sie FO2-Dateien erzeugen oder ändern können, müssen Sie die Struktur eines Arbeitsbereiches herunterladen. Sie können:
- einen leeren Arbeitsbereich herunterladen oder

- einen kompletten Arbeitsbereich mit allen Dateien herunterladen oder
- einzelne Dateien zusammen mit Ihrem Arbeitsbereich herunterladen.

Leeren Arbeitsbereich oder kompletten Arbeitsbereich mit allen Dateien herunterladen

1. Wechseln Sie in Abas Tools in das Register **PackageExplorer**.
2. Öffnen Sie das Kontextmenü eines FO2-Projects, z.B. FirstFoProject.
3. Wählen Sie den Eintrag [Abas SSH] [Download...], um das Fenster Download zu öffnen.
4. Markieren Sie den Arbeitsbereich, der heruntergeladen werden soll und klicken Sie anschließend auf **Finish**.

Dateien aus einem Arbeitsbereich herunterladen

1. Wechseln Sie in Abas Tools in das Register **PackageExplorer**.
2. Öffnen Sie das Kontextmenü eines FO2-Projects, z.B. FirstFoProject.
3. Wählen Sie den Eintrag [Abas SSH] [Download...], um das Fenster Download zu öffnen.
4. Öffnen Sie die Struktur eines Arbeitsbereiches, markieren Sie eine oder mehrere Dateien und klicken Sie anschließend auf **Finish**.

FO2-Datei

FO2-Datei erzeugen

Sie erzeugen die FO2-Datei mit einem Assistenten.

Beispiel:

Das FO2-Programm HELLO.WORLD.FO2 wird erzeugt

1. Wechseln Sie ins Register **PackageExplorer**.
2. Öffnen Sie das Kontextmenü eines Arbeitsbereiches und wählen Sie [New] [Other...].
3. Im Fenster New öffnen der Struktur des Eintrags **FO** und wählen Sie **FO2 File**. Klicken Sie anschließend auf **Next**.
4. Tragen in das Feld **File name:** den Dateinamen **HELLO.WORLD.FO2** ein und klicken Sie auf **Finish**.

Dadurch wird folgendes Programm erzeugt:

```
..!interpreter english declaration noabbrev englvar
..=====
.. Author          : bf
.. Date of Creation : 09.03.2016
.. Name            : HELLO.WORLD.FO2
.. Path/Pfad       : workdir/HELLO.WORLD.FO2
.. Function        :
.. include-easy    spx/ow
.. required Paths  :
.. Menu Items      :
.. Mask Priority    :
.. for EFOP        :
.. Starting Ticket :
..                 History of Changes
..                 =====
.. Ticket of Change :
..
..=====
..
def main(){
}
```



In der Pfadangabe müssen Sie den Arbeitsbereich (**workdir**) anpassen, z.B. **ow1**.

Die generierte FO2-Datei besteht aus:

Interpreterzeile	In der Interpreterzeile sind standardmäßig folgende Optionen definiert: • english: Kommandosprache ist Englisch • declaration: Variablen müssen vor der ersten Verwendung definiert werden • noabbrev: Kommandos und Variablennamen dürfen nicht abgekürzt werden • englvar: Englische Feldnamen werden verwendet.
Programmkopf	Hier können Sie programmspezifische Informationen eintragen. Die Informationen stammen aus der Vorlage head. Sie können die Vorlage ändern oder anpassen. • main()-Prozedur: Die main()-Prozedur wird beim Ausführen des FO2-Programms aufgerufen. In der main()-Prozedur werden Anweisungszeilen eingetragen. Die Anweisungszeilen in der main()-Prozedur werden nacheinander ausgeführt.

FO2-Datei erweitern

Beispiel:

Das FO2-Programm HELLO.WORLD.FO2 erweitern

1. Öffnen der gewünschten FO2-Datei.
2. Eintrag im Gültigkeitsbereich der main()-Prozedur von folgender Kommandozeile: **.println Hello FO2 world**. Speichern Sie anschließend die Änderung.



Sie müssen Konsolen- und Dateiausgaben mit dem Kommando **.println** realisieren.

```

..!interpreter english declaration noabbrev englvar
..=====
.. Author           : bf
.. Date of Creation : 09.03.2016
.. Name             : HELLO.WORLD.FO2
.. Path/Pfad        : ow1/HELLO.WORLD.FO2
.. Function         :
.. include-easy     spx/ow

```



```

.. required Paths      :
.. Menu Items         :
.. Mask Priority
.. for EFOP           :
.. Starting Ticket     :
..                     History of Changes
..                     =====
.. Ticket of Change    :
..
.. =====
..
def main(){
  .println Hello F02 world
}

```



Beim Speichern einer FO2-Datei wird die zugehörige FO-Datei erzeugt. Die geänderten FO2- und FO-Dateien müssen auf den Server hochgeladen werden.

Geänderte FO-/FO2-Dateien zum Server hochladen

Beispiel:

FO2-Datei HELLO.WORLD.FO2 auf den Server hochladen

1. Markieren Sie die Datei **HELLO.WORLD.FO2** und deren compilierte Datei **HELLO.WORLD.FO**.
2. Öffnen Sie das Kontextmenü in der Markierung und wählen Sie [Abas SSH] [Upload Files].

FO2-Datei (FO2-Programm) ausführen

Ein FO2-Programm wird wie ein FO-Programm ausgeführt.

Beispiel:

FO2-Datei HELLO.WORLD.FO2 ausführen

1. Öffnen Sie das Infosystem FO-Programm ausführen.
2. Entfernen Sie die Markierung im Feld "FO-Programm als Unterprogramm starten".
3. Tragen Sie in das Feld "FOP-Name oder Kommandoparameter" den voll qualifizierten Namen des FO2-Programms ein, z.B. **ow1/HELLO.WORLD.FO2** und klicken Sie auf Start.

FO2-Datei generieren



Der Compiler befindet sich auf dem Server. Die Programme werden beim Aufruf kompiliert. Die kompilierten Dateien befinden sich im Verzeichnis **fo2compiled**.

Wenn Sie die FO2-Datei manuell compilieren wollen, finden Sie die Einstellungen dazu unter: [Window] [Preferences] [Abas Tools Suite] [FO] [FO 2] [Generator] **Generated FO File extension**.

Standardmäßig ist das Feld mit **.FO** vorbelegt. Damit vorhandene FO-Dateien mit gleichem Bezeichner nicht überschrieben werden, verwenden Sie z.B. **.GEN.FO**.

Editoreigenschaften

VAR-Tag

- Definieren Sie die Variablen in einem VAR-Tag.
- Sie können ein VAR-Tag mit dem Minuszeichen zuklappen und mit dem Pluszeichen aufklappen.
- Variablen, die im Code verwendet werden ohne definiert zu sein, können Sie mit der Tastenkombination **Ctrl + 1** im VAR-Tag implementieren.
- Die Option **folded** bewirkt, dass das Tag beim Öffnen der Datei zugeklappt dargestellt wird.

Beispiel:

VAR-Tag, beim Öffnen der Datei aufgeklappt

```
..<VAR>  
userdefined code  
..</VAR>
```

Beispiel:

VAR-Tag, beim Öffnen der Datei zugeklappt

```
..<VAR folded>  
userdefined code  
..</VAR>
```

Beispiel:

VAR-Tag in dem eine Variable vom Typ text und dem Bezeichner criteria definiert ist

```
..<VAR folded>  
.type text criteria ? F|defined(U|criteria) = G|false  
..</VAR>
```

BLOCK-Tag

- Sie können um einen Codebereich ein BLOCK-Tag definieren.
- Sie können ein BLOCK-Tag mit dem Minuszeichen zuklappen und mit dem Pluszeichen aufklappen.
- Die Option **folded** bewirkt, dass das Tag beim Öffnen der Datei zugeklappt dargestellt wird.

Beispiel:**BLOCK-Tag, beim Öffnen der Datei aufgeklappt**

```
..<BLOCK>  
userdefined code  
..</BLOCK>
```

Beispiel:**BLOCK-Tag, beim Öffnen der Datei zugeklappt**

```
..<BLOCK folded>  
userdefined code  
..</BLOCK>
```

Beispiel:**Definierter Codebereich wird mit BLOCK-Tag begrenzt. Beim Öffnen des Programms wird der BLOCK zugeklappt dargestellt.**

```
..<BLOCK folded>  
.type text criteria ? F|defined(U|criteria) = G|false  
..</BLOCK>
```

TODO

Die TODO-Eigenschaft wird in einer Kommentarzeile verwendet, um eine Aufgabe zu definieren. Alle TODOs werden im Register **Tasks** aufgelistet. Sie können pro Kommentarzeile ein TODO definieren. Das TODO wird im Register **Tasks** in einer separaten Zeile eingetragen.

```
.. TODO first read head field  
.. TODO next read table field
```

FIXME

Die FIXME-Eigenschaft wird in einer Kommentarzeile verwendet, um ein erkanntes Problem zu markieren. Alle FIXMEs werden im Register **Tasks** mit einem Ausrufezeichen aufgelistet. Sie können pro Kommentarzeile ein FIXME definieren. Das FIXME wird im Register **Tasks** in einer separaten Zeile

eingetragen.

```
.. FIXME solve this problem  
.. FIXME solve that problem
```

META-Tag

Mit META-Tags werden Puffer initialisiert. Dadurch werden über den Pufferbuchstaben, gefolgt vom | - Zeichen, die Felder der definierten Datenbankgruppe oder des Infosystems zur Auswahl angeboten.

Wenn Datensätze der Datenbankgruppe Kunde mit dem Kommando `.select` selektiert wurden, dann müssen Sie den H-Puffer wie folgt initialisieren, damit die Felder der Gruppe Kunde über den H-Puffer zur Auswahl angeboten werden: `..`

Wenn das Infosystem `ow1/VARNAMELIST` verwendet wird, dann müssen Sie den M-Puffer wie folgt initialisieren, damit die Felder des Infosystems über den M-Puffer zur Auswahl angeboten werden: `..`

Wenn Datensätze der Datenbankgruppe Artikel mit dem Kommando `.load 0` geladen wurden, dann müssen Sie den 0-Puffer wie folgt initialisieren, damit die Felder der Gruppe Artikel über den 0-Puffer zur Auswahl angeboten werden: `..`



Sie müssen jeden Puffer nach der beschriebenen Vorgehensweise initialisieren, damit die Felder des Datensatzobjekts, über den Pufferbuchstaben zur Auswahl angeboten werden.

Wenn sich in einem Puffer, nach einer Selektion, Datensätze einer anderen Datenbankgruppe befinden, dann müssen Sie den Puffer entsprechen initialisieren.

FO2-Sprachkomponenten

Kommentare

- Kommentarzeilen beginnen mit 2 aufeinanderfolgenden Punkten.
- Mit der Tastenkombination Strg + 7 können Sie eine oder mehrere markierte Zeile(n) auskommentieren oder die Auskommentierung entfernen.

Beispiel:

Definition einer Kommentarzeile

```
.. this is a comment
```

Variablen

- Variablen werden vor der main()-Prozedur definiert und initialisiert.
- Variablen können mit den Kommandos `.type` und `.var` definiert werden.
- Definieren Sie die Variablen in einem `VAR-Tag`.

Beispiel:

Variablen deklarieren

Definieren einer Variable vom Typ `integer` mit dem Bezeichner `number` und Initialisierung dieser Variable mit dem Literal 12.

Definieren einer Variable vom Typ `integer` mit dem Bezeichner `counter`, falls diese Variable noch nicht existiert.

```
.var integer number = 12  
.type integer counter ? F|defined(U|counter) = G|false
```

Prozeduren

- Jedes FO2-Programm besitzt eine Prozedur mit dem Bezeichner `main()`.
- In die `main()`-Prozedur können Sie Prozeduraufrufe einbauen.
- Eine Prozedur wird über das Kommando `.call` aufgerufen.
- Wenn Sie nach `.call` den Prozeduraufruf eintragen, dann können Sie die zugehörige Prozedur generieren lassen.

Beispiel:

Prozedur aufrufen

In der Prozedur `main()` wird mit `.call`, die Prozedur `displayInfo()` aufgerufen. Die zugehörige Prozedur wird generiert. Beim Ausführen der Datei wird die Prozedur aufgerufen und der Inhalt der Text-Variablen `info` ausgegeben.

```
.var text info = "procedure called"
def main(){
    .call displayInfo()
}
..
def displayInfo( ) {
    .println 'U|info'
}
```

if-Bedingungen

Beispiel:

Syntax

Die if-Bedingung wird verwendet, um zu prüfen ob eine Bedingung erfüllt ist oder nicht. Wenn die Bedingung erfüllt ist, dann wird das Programm im Gültigkeitsbereich des if-Bereichs fortgesetzt, ansonsten im Gültigkeitsbereich des else-Bereichs.

```
if(expression = G|true) {
    .. to this
}else{
    .. to that
}
```

Beispiel:

CHECK.SELECTED.CUSTOMER

In der FO2-Datei `CHECK.SELECTED.CUSTOMER` wird ein Kunde mit einer definierten Identnummer selektiert. Mit der if-Bedingung wird geprüft, ob der Kundendatensatz geladen werden konnte.

Falls ja, wird der Text `customer available` mit dem Suchwort und der Bezeichnung ausgegeben.

Falls nein, wird der Text `customer 'H|idno' NOT available` ausgegeben.

```
..
```

```
def checkCustomerRecord( ) {
  ..<META H|= 'P0:1'>
  if(G|success = G|true) {
    .println customer available -> 'H|swd' 'H|descrOperLang'
  }else{
    .println customer 'H|idno' NOT available
  }
}
```

switch-case

Beispiel:

Syntax

Mit der switch-case-Anweisung können Sie mehrfach verzweigen, abhängig vom Wert einer Variablen. Im switch-Zweig sind Variablen aller Abas Typen zulässig. Wenn der Inhalt eines case-Zweiges abgearbeitet wurde, dann wird das Programm nach der switch-case-Anweisung fortgesetzt. Wenn kein case-Zweig abgearbeitet wurde, dann wird der default-Zweig abgearbeitet.

```
switch(U|input){
  case("x"){
    .. userdefined code
  }
  case("y"){
    .. userdefined code
  }
  case("z"){
    .. userdefined code
  }
  default{
    userdefined code
  }
}
```

Beispiel:

USE.SWITCH.CASE.FO2

Im FO2-Programm **USE . SWITCH . CASE . FO2** soll der Benutzer die zu selektierende Gruppe eingeben (Customer, Product, Invoice, usw.). Die Eingabe wird in der Textvariable **U|input** gespeichert. Entsprechend der Eingabe (Customer, Product, Invoice, usw.) wird in switch-case verzweigt. Im case-Zweig erfolgt der zugehörige Prozeduraufruf. In der Prozedur erfolgt die Ausgabe select '**U|input**'.

```
..<VAR folded>
  .var text input = ""
..</VAR>
..
```



```
def main(){
    .read "Insert group to select" U|input
    if(U|input <> "") {
        .call getSelection()
    }else{
        .println no group defined
        .return
    }
}

def getSelection( ) {
    switch(U|input){
        case("Customer"){
            .call displayCustomers()
        }
        case("Product"){
            .call displayProducts()
        }
        case("Invoice"){
            .call displayInvoices()
        }
        default{
            .println 'U|input' should be selected
        }
    }
}

def displayCustomers( ) {
    .println select 'U|input'
}

def displayProducts( ) {
    .println select 'U|input'
}

def displayInvoices( ) {
    .println select 'U|input'
}
```

while

Mit der while-Schleife wird der Programmcode im Gültigkeitsbereich, in Abhängigkeit einer Bedingung wiederholt. Solange die Bedingung erfüllt ist, wird der Programmcode im Gültigkeitsbereich der while-Schleife ausgeführt. Wenn die Bedingung nicht mehr erfüllt ist, dann wird das Programm nach der while-Schleife fortgesetzt.

Beispiel:

Syntax

```
while(Expression = G|true){  
    .. userdefined code  
}
```

Die while-Schleife wird fünfmal durchlaufen. In jedem Schleifendurchlauf wird der Text **Schleifendurchlauf: 'U|index'** ausgegeben.

```
...var integer index = 1  
.var integer length = 5  
..def main(){  
    while(U|index <= U|length){  
        .println Schleifendurchlauf: 'U|index'  
        .formula U|index = U|index + 1  
    }  
    .println end of program  
}
```

do while

Mit der do while-Schleife wird der Programmcode im Gültigkeitsbereich, in Abhängigkeit einer Bedingung wiederholt. Bei einer do while-Schleife wird der Programmcode mindestens einmal durchlaufen, erst danach findet die Prüfung statt. Solange die Bedingung erfüllt ist, wird der Programmcode im Gültigkeitsbereich der do while-Schleife ausgeführt. Wenn die Bedingung nicht mehr erfüllt ist, dann wird das Programm nach der do while-Schleife fortgesetzt.

Beispiel:

Syntax

```
do{  
    .. userdefined code  
}while(Expression = G|true)
```

Die while-Schleife wird mindestens einmal und max. fünfmal durchlaufen. In jedem Schleifendurchlauf wird der Text **Schleifendurchlauf: 'U|index'** ausgegeben.

```
...var integer index = 1  
.var integer length = 5  
..
```

```
def main(){  
  do {  
    .println Schleifendurchlauf: 'U|index'  
    .formula U|index = U|index + 1  
  } while(U|index <= U|length )  
  .println end of program  
}
```

.break

Das Kommando **.break** wird verwendet, um eine Schleife abubrechen.

Die while-Schleife wird definiert verlassen, wenn der Wert der Variable **index = 4** ist.

```
...var integer index = 1  
.var integer length = 10  
.var integer break = 4  
..def main(){  
  while(U|index <= U|length){  
    if(U|index = U|break ) {  
      .break  
    }  
    .println Schleifendurchlauf: 'U|index'  
    .formula U|index = U|index + 1  
  }  
  .println end of program  
}
```

.continue

Das Kommando **.continue** wird verwendet, um den nächsten Schleifendurchlauf auszulösen.

Die while-Schleife wird durchlaufen. Nur bei ungeradem Wert der Variable **index** wird dieser ausgegeben.

```
...var integer index = 1  
.var integer length = 10  
.var integer break = 4  
.type integer result  
..def main(){
```

```
while(U|index <= U|length){
    .formula U|result = U|index // 2
    .formula U|index = U|index + 1
    if(U|result <> 0 ) {
        .continue
    }
    .println Schleifendurchlauf: 'U|index'
}
.println end of program
}
```

.return

Das Kommando **.return** wird verwendet, um eine Schleife definiert zu beenden. Anschließend wird zum aufrufenden Programm zurückgekehrt.

Die while-Schleife wird durchlaufen. Nach einer definierten Anzahl von Durchläufen (**loopEnd**) verlassen Sie mit **.return** die Schleife und beenden das FO2-Programm.

```
..
```

Templates

Sie können im Programmcode von Abas gelieferte Standardtemplates verwenden. Alternativ können Sie auch eigene Templates erzeugen und im Programmcode verwenden. Die Liste aller Templates erhalten Sie mit STRG+LEERTASTE. Wenn Sie eine neue FO2-Datei erzeugen, dann wird das Template **head** zur Anlage des Dateikopfes verwendet.

Um Templates zu verwenden, tragen Sie im Quellcode die Anfangsbuchstaben des Templatenamens ein und drücken Sie die Tastenkombination STRG+LEERTASTE. Eine Auswahlliste öffnet sich. Wählen Sie das gewünschte Template aus und der Code des Templates wird an der Position des Cursors eingebaut.

Eine Übersicht aller Templates in Abas Tools erhalten Sie über [Window] [Preferences] [Abas Tools Suite] [Fop2] [Templates].

Beispiel:

Erzeugen eines benutzerdefinierten Templates

Ein Template für eine while-Schleife wird erzeugt. Die Bedingung ist **G|success = G|false**. Der Cursor wird im Gültigkeitsbereich positioniert.

1. Wählen Sie [Window] [Preferences] [Abas Tools Suite] [Fop2] [Templates] um das Fenster Preferences zu öffnen.
2. Klicken Sie auf **New...**, um das Fenster New Template zu öffnen.
3. Füllen Sie folgende Felder aus:
Name: **while condition**
Context: BodyContent
Description: **G|success = G|true**
Pattern:

```
while(G|success = G|false){  
  ${cursor}  
}
```

+ . Klicken Sie **Ok**, um das Template zu speichern.

Beispiel:

Verwenden eines benutzerdefinierten Templates

Verwendung des Templates **while condition**.

1. Positionieren Sie den Cursor an der gewünschten Stelle und drücken Sie STRG+LEERTASTE.
2. Führen Sie einen Doppelklick auf die Vorlage **while condition** aus.

Datensätze selektieren

Sie können Datensätze mit oder ohne Selektionsstring selektieren. Der Selektionsstring wird in einer Textvariablen gespeichert. Mit einer while-Schleife iterieren Sie über die selektierten Datensätze. Über die Tabellenzeilen iterieren Sie mit einer inneren do-while-Schleife.

Kundendatensätze selektieren

Kundendatensätze werden in einem definierten Identnummernbereich selektiert. Die Datei **SELECT.CUSTOMER.CRITERIA.F02** wird erzeugt. Mit einem Selektionsstring werden Kundendatensätze in einem definierten Identnummernbereich selektiert. Die benötigten Variablen werden definiert und initialisiert. Selektion und Iteration werden über die selektierten Datensätze ausgeführt.

```
..<VAR folded>
.type text criteria ? F|defined(U|criteria) = G|false
.type text read ? F|defined(U|read) = G|false
..</VAR>
..
def main(){
  .formula U|criteria = "$,idno=10004!10008;swd=;@group=1;@language=en"
  .. Selektion mit Kriterien ausführen -> Resultset wird ermittelt
  .select customer 'U|criteria'
  while(G|success = G|true){
    .call displayHeaderFields()
    .select continue
  }
}
def displayHeaderFields( ) {
  ..<META H|= 'P0:1'>
  .println 'H|idno' 'H|swd' 'H|descrOperLang' 'H|zipCode'
}
```

Artikeldatensätze selektieren

Artikeldatensätze werden in einem definierten Identnummernbereich selektiert. Die Datei **SELECT.PRODUCTS.LINES.F02** wird erzeugt. Mit einem Selektionsstring selektieren Sie Artikeldatensätze in einem definierten Identnummernbereich. Die Bereichsgrenzen werden vom Benutzer eingeben. Kopf- und Tabellenfeldinhalte werden ausgegeben. Die benötigten Variablen werden definiert und initialisiert. Iterieren Sie mit einer while-Schleife über die Datensätze und mit einer do-while-Schleife über die Tabellenzeilen des aktuellen Datensatzes.

```
..<VAR folded>
.. Variablen definieren
.type text criteria ? F|defined(U|criteria) = G|false
.type text from ? _F|defined(U|from)
```

```

.type text to ? _F|defined(U|to)
.. Variablen definieren und initialisieren
.var text instructionFrom = "Identnummer (von) eingeben:"
.var text instructionTo = "Identnummer (bis) eingeben:"
..</VAR>
..
def main(){
  .call readIdnoRange()
  .formula U|criteria = "$,idno=" + 'U|from' + "!" + 'U|to' + ";swd=;@group=1;@language=en"
  .select parts 'U|criteria'
  while(G|success = G|true){
    .call displayHeaderFields()
    do {
      .select line
      if(G|success = G|true) {
        .call displayLineFields()
      }else{
        .break
      }
    } while(G|success = G|true)
    .select continue
  }
  .println program end
  .end
}

def readIdnoRange( ) {
  .read 'U|instructionFrom' U|from
  .read 'U|instructionTo' U|to
}

def displayLineFields( ) {
  ..<META H|= 'P2:1'>
  .println ++ 'H|productListElem^idno' 'H|productListElem^swd' 'H|elemQty'
}

def displayHeaderFields( ) {
  ..<META H|= 'P2:1'>
  .println 'H|idno' 'H|swd' 'H|salesPrice'
}

```

Menü verwenden

In einem Menüfenster werden mehrere Menüelemente dargestellt.

Beispiel:

Menü mit Einfachauswahl

Die Datei **USE.MENU.0.F02** und ein Menü mit 3 Menüelementen wird erzeugt. Nach Auswahl eines Menüelements wird das Menüfenster geschlossen. Der Index und der Text des gewählten Menüpunkts werden ausgegeben.

```
def main(){
  .menu Select Item
  .item item1 => "First menu item"
  .item item2 => "Second menu item"
  .item item3 => "Third menu item"
  .endItem

  if(G|noSelectedMenuOption <> 0) {
    .println selected menu item: 'G|noSelectedMenuOption' -> 'G|menuText'
  }else{
    .println menu canceled
  }
  .println continue program
}
```

Beispiel:

Menü mit Mehrfachauswahl

Das Menüfenster bleibt geöffnet, bis es geschlossen wird. Dazu wird eine do-while-Schleife verwendet. Nach Auswahl eines Menüelements wird die zugehörige Prozedur ausgeführt.

```
def main(){
  do {
    .. Menü definieren
    .menu Select Group
    .item "group1"    =>.call procedur1()
    .item "group2"    =>.call procedur2()
    .item "group3"    =>.call procedur3()
    .endItem
  } while(G|noSelectedMenuOption <> 0)
  .println continue program
}

def procedur1( ) {
  .println run group1
}

def procedur2( ) {
  .println run group2
}
```



```
}  
def procedur3( ) {  
    .println run group3  
}
```

Textbox

In einer Textbox werden Informationen ausgegeben. Der Benutzer muss die Textbox bestätigen. Dafür stehen die Buttons Yes, No, Cancel und Ok zur Verfügung. Standardmäßig besteht eine TextBox aus Überschrift, Text und Button Ok.

Beispiel:

Textbox Standard

Die Datei **USE.BOX.0.F02** wird erzeugt. Die Standardtextbox mit Überschrift, Text und Button Ok wird erzeugt.

Das Programm wird nach dem Schließen der Textbox fortgesetzt.

```
def main(){
  .box Hint
  .item "Selection accessed"
  .enditem
  .println continue program
}
```

Beispiel:

Textbox mit Buttons

Die Datei **USE.BOX.2.F02** wird erzeugt. Eine benutzerdefinierte Textbox mit mehrzeiligem Text und den Buttons Yes und No wird erzeugt. Nach dem Klicken des Buttons erfolgt eine Abfrage. Die entsprechende Meldung wird ausgegeben.

```
def main(){
  .box -BUTTON yes:no -TITLE Hint
  .item "Log changes?"
  .item "(Yes / No)"
  .enditem
  if(G|box = G|boxyes){
    .println Log changes
  }else{
    .println Do not log changes
  }
  .println continue program
}
```

etf-Datei importieren

Um den Umstieg von EasyCODE zu Abas Tools zu erleichtern, können Sie von EasyCODE erzeugte ETF-Dateien in die Abas Tools importieren. Dabei wird die ETF-Datei ins FO2-Format gewandelt.

Import von ETF-Dateien in Abas Tools

ETF-Dateien müssen in ein Verzeichnis eines FO2-Projekts importiert werden.

1. Öffnen Sie das Kontextmenü auf einem Verzeichnis und wählen Sie den Eintrag **Import...** um das Fenster Import zu öffnen.
2. Öffnen Sie die Struktur des Eintrags **EasyCODE** und markieren Sie den Eintrag **Import ETF Files**.
3. Klicken Sie auf **Next**, um das Fenster **File system** zu öffnen.
4. Tragen Sie im Feld **From directory:** den Pfad zur gewünschten ETF-Datei ein oder wählen Sie diesen mit **Browse**.
5. Definieren Sie die Interpreteroptionen im Feld **Interpreter Line:** und passen Sie diesen gegebenenfalls an.
6. Definieren Sie das Encoding (z.B. windows-1250) im Feld **ETF Files Charset**. Das Encoding der ETF-Datei finden Sie in EasyCODE in der zugehörigen Konfig-Datei (*.cfg).
7. Starten Sie den Import mit Klick auf **Finish**.

Diff's

Beim Upload-Prozess wird geprüft, ob die Remote-Datei auf dem Server geändert wurde. Falls ja, wird ein Hinweis ausgegeben. Der Upload-Prozess kann fortgesetzt oder abgebrochen werden. Wenn abgebrochen wird, dann findet kein Upload statt. Kennzeichen für unterschiedliche Dateiinhalte (Diff) ist die rote Raute.

Diff anzeigen

1. Öffnen Sie das Kontextmenü für eine mit roter Raute markierte Datei.
2. Wählen Sie [FOP Source] [Compare with Server] um im Code-Bereich das Register Text Compare zu öffnen. Die Diffs werden hervorgehoben.
3. Mit Klick auf das Pfeil-Symbol im Mittelsteg wird der markierte Bereich von der Remote-Datei zur Local-Datei übertragen.
4. Speichern Sie die Änderungen und laden Sie die geänderte Datei auf den Server.

Weitere Beispiele

Array in FO2 verwenden

Im Beispiel wird ein Array vom Typ Text mit 10 Elementen verwendet. Sie müssen das Array vor der Verwendung des 1. Elements initialisieren. Im Programm können Sie jedes Array-Element über einen Index vom Typ Integer ansprechen.

```
..
```

Zugriff auf die Variable actionname (deutsch: kaktname)

Im Beispiel wird über die Variable **G|evtCmd** (deutsch: **G|evtkommd**) mit einer switch-case-Anweisung, verzweigt. Im case-Zweig wird mit der Punktersetzung, auf den Inhalt der Variablen **G|actionname**

(deutsch: **G|kaktname**) reagiert, z.B. **G|actionname12**, **G|actionname13** oder **G|actionname14**.

```
..<VAR folded>
.var int index12 = 12
.var int index13 = 13
.var int index14 = 14
..</VAR>

def main(){
  switch(G|evtCmd){
    case(G|actionName.U|index12){
      .println -view-
    }
    case(G|actionName.U|index13){
      .println -edit-
    }
    case(G|actionName.U|index14){
      .println -new-
    }
    default{
      .println something else
    }
  }
}
```